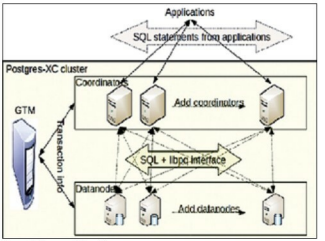

Figure 1: Postgres-XC architecture

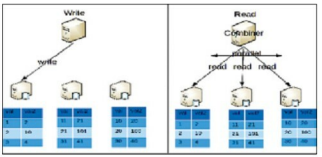

Figure 2: Distributed tables

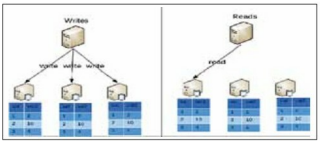

Figure 3: Replicated tables

manages information about transactional activities in Postgres-XC, issues global transaction identifiers to transactions (to maintain a consistent view of the database on all nodes), and provides ACID properties. It provides support for other global data, such as sequences and time-stamps. It stores no user data, except control information.

2. *Coordinators* (masters) provide a point of contact for the application/client. They are responsible for parsing and executing queries from the clients, and returning the results (if needed). They do not store any user data themselves, but gather the data from datanodes, with the help of SQL queries fired through a PostgreSQL-native interface. The coordinators also process the data if required and even manage the two-phase commit. Although coordinators do not store user data, they use the catalogue data to parse queries, resolve symbols, plan queries, locate data, etc.
3. *Datanodes* store user data and catalogues. The datanodes execute the queries received from the coordinator and return results to the coordinator.

Distribution of data and scalability

Postgres-XC allows two ways of storing the tables on the datanodes:

1. **Distributed tables:** A table is distributed on a given set of datanodes using strategies like hash, round-robin, or modulo partitioning. Every row of a distributed table resides on a single datanode. Multiple rows can be modified or written in parallel to various datanodes; we can also read the rows from various datanodes in parallel. Performance is greatly improved by parallel writes and reads from different datanodes.
2. **Replicated tables:** A table is replicated on a given set of datanodes using statement-level replication, which performs better than log-based replication, since the size of the logs that must be shipped is much greater than the size of the statement. In the case of a replicated table, a row in the table resides on each datanode on which the table is replicated. Any modifications to the row must be duplicated to each replicated copy. Since all the data in the table is available on a single datanode, the coordinator can gather all the data from a single node and in some cases, act as a proxy between the client application and the datanode. This allows multiple queries on the same table to be directed to different datanodes, thus balancing the load and increasing the read throughput.

Figures 2 and 3 depict the read and write concepts for distributed and replicated tables, respectively.

High availability

To achieve high availability, one needs data redundancy, component redundancy and automatic failover. In Postgres-XC, data redundancy can be achieved by using the PostgreSQL native replication with Hot Standby for datanodes. Since each coordinator is a master (capable of writing data) and is capable of reading writes performed by any other coordinator instantaneously, every coordinator is capable of replacing any other, should that coordinator fail. GTM-standby acts as a redundant component for GTM. However, third-party tools are required for automatic fail-over of all the three types of components.

Performance evaluation

Initial transaction throughput measurements carried out using the DBT-1 benchmark have shown significant throughput scalability, as shown in Figure 4. The figure plots the Scaling Factor versus the Number of Servers in Postgres-XC. The Scaling Factor is the ratio of the number